

Teamwork III (DevOps, Best Practices and Tools)

Group members: Sanuji Wimansa Gamagedara e2401855,

Yashoda Buthpitiya e2403004,

Samurdhika Susanthi e2401866

Questions

1. What is the main purpose of DevOps?

The goal of DevOps is to improve collaboration between development and operations teams by automating and simplifying the processes of building, testing, releasing, and maintaining applications.

2. What are the main stages of a CI/CD pipeline?

- **Code** – developers write code.
- **Build** – the application is compiled.
- **Test** – automated tests check the code.
- **Release** – the software is prepared for deployment.
- **Deploy** – the software is deployed to an environment.
- **Monitor** – the application is monitored after deployment.

3. Which DevOps tools can be used for source control, CI/CD, containerization, infrastructure, configuration management, and monitoring?

DevOps Area	Tools
Source Control	Git, GitHub, GitLab, Bitbucket
CI/CD	Jenkins, GitHub Actions, GitLab CI/CD, Azure DevOps
Containerization	Docker, Podman
Infrastructure	Terraform, AWS CloudFormation
Configuration Management	Ansible
Monitoring	Prometheus, Grafana

4. How can Git support effective collaboration in DevOps?

Developers can create branches for features or fixes, commit changes, and merge work back into shared branches.

5. What is Infrastructure as Code (IaC), and what are its benefits?

managing and provisioning infrastructure using code and automation rather than manual processes. It involves version controlling the infrastructure configurations just like application code, making deployments repeatable and consistent.

benefits:Automation, consistence, version control, repeatability, faster deployments

6. What is configuration management, and how can Ansible support it?

Configuration management means managing and keeping servers and systems configured correctly and consistently.

Ansible is primarily used for configuration management and automation. It can automate tasks such as:

- Installing software
- Configuring servers
- Creating users
- Deploying applications
- Applying consistent settings across multiple machines

7. What are containers, and why are they useful in DevOps?

containers are lightweight , isolated environments that package an application together with the dependencies it needs to run.

Containers help reduce differences between development, testing, and production environments.

8. What is container orchestration, and what role does Kubernetes play?

container orchestration is automated management of containers across one or more machines.

Deploys, scales, schedules, and manages containerized workloads across clusters

9. What are monitoring and observability, and which tools support them?

ongoing monitoring of systems, applications, and infrastructure to ensure health, performance, and security. It involves setting up monitoring, logging, and alerting mechanisms to detect issues in real time. Tools: Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana).

10. What is DevSecOps, and how does it improve software security?

embedding security practices throughout the development process. It involves integrating security checks in CI/CD pipelines (shift-left security) to detect vulnerabilities earlier.

11. Why is automated testing important in DevOps?

Automated testing is important in DevOps because it finds bugs early and ensures that new code works correctly with existing code.

12. What is configuration drift, and how can it be reduced?

Configuration drift is when servers or systems become different from their intended configuration over time.

It can be reduced by using automation and configuration management tools like Ansible to apply the same settings consistently across systems.

13. Why are small and frequent deployments considered a DevOps best practice?

Small and frequent deployments make software delivery faster, safer, and more reliable.

14. What are the main steps for responding to a failed deployment?

- Assess the failure
- stop the deployment
- investigate the root cause
- fix and test
- redeploy
- documentation

15. How can Git, Jenkins, Docker, Kubernetes, Terraform, Ansible, Prometheus, and Grafana work together?

- Git → source-code management
- Jenkins → build/test/deployment automation
- Docker → application containerization
- Kubernetes → container orchestration
- Terraform → infrastructure provisioning
- Ansible → configuration automation
- Prometheus → metrics collection and alerting
- Grafana → monitoring dashboards